

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

STUDENT NAME: PRIYANKA .G

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 2

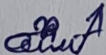
Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Priyanka G/192524124 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

92



Annexure A

Constraint-Based Problem Solving

Annexure A

Student 31: Priyanka G | Register Number: 192524124

1. A courier company must assign 3 delivery trucks (Truck1, Truck2, Truck3) to 3 delivery time-windows (9-11 AM, 11 AM-1 PM, 1-3 PM) such that:

Constraints:

- i. Truck1 cannot deliver in the 1-3 PM window
- ii. Truck2 must deliver before Truck3 ($9-11 < 11-1 < 1-3$)
- iii. Only one truck per time-window
- iv. Truck3 cannot deliver in the 9-11 AM window
- v. All trucks must be assigned exactly one time-window

Apply Backtracking Search to find a valid assignment, showing all intermediate steps and constraint checks. State the final valid schedule.

2. A warehouse maze-solving robot operates in a grid environment (5x5) with shelf obstacles. It must move from Start (S) to Goal (G).

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- iv. Cost of each move = 1
- iii. Cannot pass through shelf (obstacle) cells
- iv. Use Manhattan Distance heuristic for guidance

Using the above constraints and A* Search, show step-by-step node expansion, $f(n)=g(n)+h(n)$ values at each step, and determine the optimal path and cost.

DATE: 20/07/26.

CSA717A ARTIFICIAL INTELLIGENCE

NAME: PRIYANKA.G

CO1 ASSESSMENT TOOL - 2

REG. NO: 192524124

CONSTRAINT BASED
PROBLEM SOLVING.

1) Constraint Based Problem solving using Backtracking search.

→ Step 1: Defining the variables

Variable	Meaning
T_1	Truck 1
T_2	Truck 2
T_3	Truck 3

→ Step 2: Domains

$$T_1 = \{9-11, 11-1\}$$

$$T_2 = \{9-11, 11-1, 1-3\}$$

$$T_3 = \{11-1, 1-3\}$$

⇒ Step 3: Constraints

$$C_1: \text{Truck 1} \neq 1-3 \text{ PM}$$

$$C_2: \text{Truck 2} < \text{Truck 3}$$

C_3 : Every slot has only one truck

$$C_4: \text{Truck 3} \neq 9-11 \text{ AM}$$

C_5 : Every truck gets exactly one slot

Time order

9-11

↓

11-1

↓

1-2

⇒ Step 4: Backtracking Search

> Initial Assignment: $\{\}$

> Assign Truck 1: Choose first legal value: $T_1 = 9-11$
current Assignment: $\{T_1 = 9-11\}$.
constraint check: Valid.

> Assign Truck 2: • Try $T_2 = 9-11$

conflict: same slot already occupied → Reject
Backtrack.

• Try next value: $T_2 = 11-1$

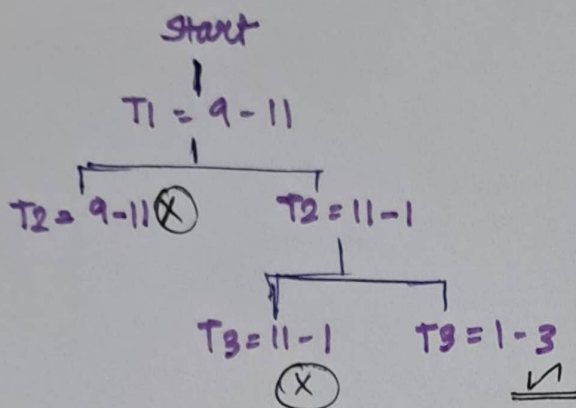
Current Assignment: $T_1 = 9-11$; $T_2 = 11-1$.

constraint valid

Assign Truck 3

- Try $T_3 = 11-1$
conflict ahead assigned - Reject
Back track
- Try $T_3 = 1-3$
constraint check: Truck 3 $\neq$ 9-11
Truck 2 before Truck 3
Unique slot - All trucks assigned

Search Tree



Constraint Verification.

- Truck 1 not in 1-3 ✓
- Truck 2 before Truck 3 ✓
- Truck 3 not in 9-11 ✓
- One truck per slot ✓
- Every truck assigned ✓

Conclusion: Final assignment: Backtracking search successfully finds the valid schedule:

Truck 1 $\rightarrow$ 9-11 AM

Truck 2 $\rightarrow$ 11 AM - 1 PM

Truck 3 $\rightarrow$ 1-3 PM.

Backtracking Search is an uniformed search algorithm used to solve Constraint Satisfaction Problems (CSP-s)

PSEUDO CODE:

Backtracking (variable assignment)

If all variables are assigned
return

select an unassigned variable

For each possible value

assign value

If constraints are satisfied - continue recursively

Else Backtrack

Return Failure.

2) Warehouse Maze using A* Search Algorithm

Assumed 6x5 Grid:

S = Start

G = Goal

X = Obstacle

• = Empty cell

	C1	C2	C3	C4	C5
R1	S	•	•	X	•
R2	X	X	•	X	•
R3	•	•	•	•	•
R4	•	X	X	X	•
R5	•	•	•	•	G

⇒ Movement Rule.

Up (✓), Down (✓), Left (✓), Right (✓) : Cost = 1.

→ Cannot cross obstacle.

Heuristic: $h(n) = |\text{Goal Row} - \text{Current Row}| + |\text{Goal Column} - \text{Current Column}|$

→ Manhattan Distance

⇒ Step 1:

> Start node: $S = (1,1)$

: Goal: $G = (5,5)$.

⇒ Step 2:

Initial calculation $g = 0$

$$h = |5 - 1| + |5 - 1| = 8$$

$$f = g + h$$

$$f = 8$$

⇒ Step 3: Node Expansion Table

Step	Node	$g(n)$	$h(n)$	$f(n)$
1	(1,1)	0	8	8
2	(1,2)	1	7	8
3	(1,3)	2	6	8
4	(2,3)	3	5	8
5	(3,3)	4	4	8
6	(3,4)	5	3	8
7	(3,5)	6	2	8
8	(4,5)	7	1	8
9	(5,5)	8	0	8

Expanded Nodes

(1,1)
↓
(1,2)
↓
(1,3)
↓
(2,3)
↓
(3,3)
↓
(3,4)
↓
(3,5)
↓
(4,5)
↓
(5,5)

> Optimal Path:

$S \rightarrow (1,2) \rightarrow (1,3) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (3,5) \rightarrow (4,5) \rightarrow G$

> Path Representation:

$S \rightarrow \rightarrow \downarrow \downarrow \rightarrow \rightarrow \downarrow G$

> Total cost:

Each move = 1 : Total moves = 8 : Optimal Cost = 8.

→ A* Search Tree:

$S - (1,2) - (1,3) - (2,3) - (3,3) - (3,4) - (3,5) - (4,5) - G$

⇒ Conclusion:

Using A* Algorithm with the Manhattan Distance heuristic, the robot successfully reaches the goal while avoiding obstacles. The optimal path contains 8 moves, giving minimum path cost of 8.

A* Algorithm Search is an informed search algorithm that finds the shortest and most optimal path from a start node to a goal node. It combines the actual path from the start with a heuristic estimate the distance using " $f_n = g(n) + h(n)$ ".

Manhattan distance heuristic distance: $h(n) = |x_g - x_n| + |y_g - y_n|$.

Working Principle:

Start the initial node

Calculate $g(n)$, $h(n)$, $f(n)$

Add neighboring nodes to OPEN list.

Select the node with lowest $f(n)$.

Move the selected node to the CLOSED list.

Repeat until the Goal node is reached.

Track back the optimal path.